西南交通大學
Southwest Jiaotong University

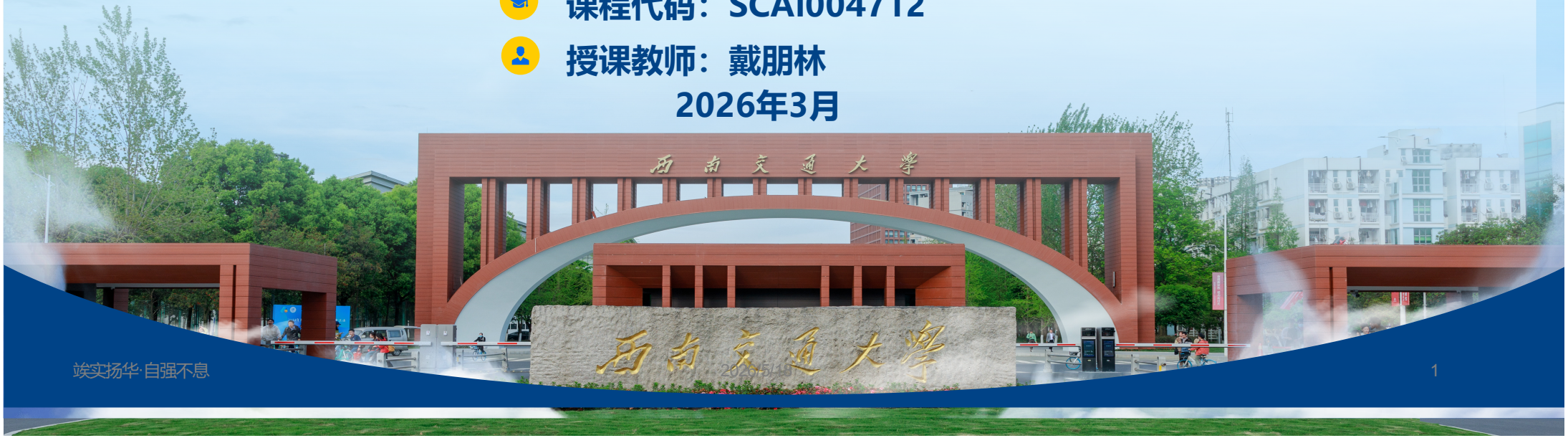
云计算技术

第9讲 并行编程模型

课程代码：SCAI004712

授课教师：戴朋林

2026年3月



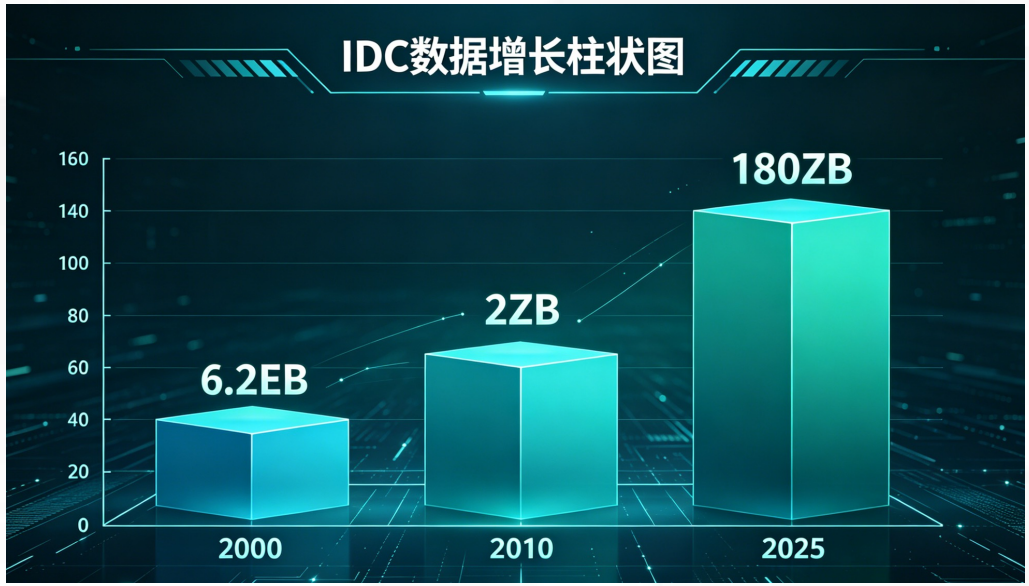
00 本讲内容大纲

西南交通大学 Southwest Jiaotong University

1. 并行计算基础
2. MapReduce编程模型
3. MapReduce执行流程
4. MapReduce编程模式
5. Hadoop MapReduce实践
6. MapReduce的局限与演进
7. 应用案例与演示代码

1 并行计算基础

01 为什么需要并行计算?





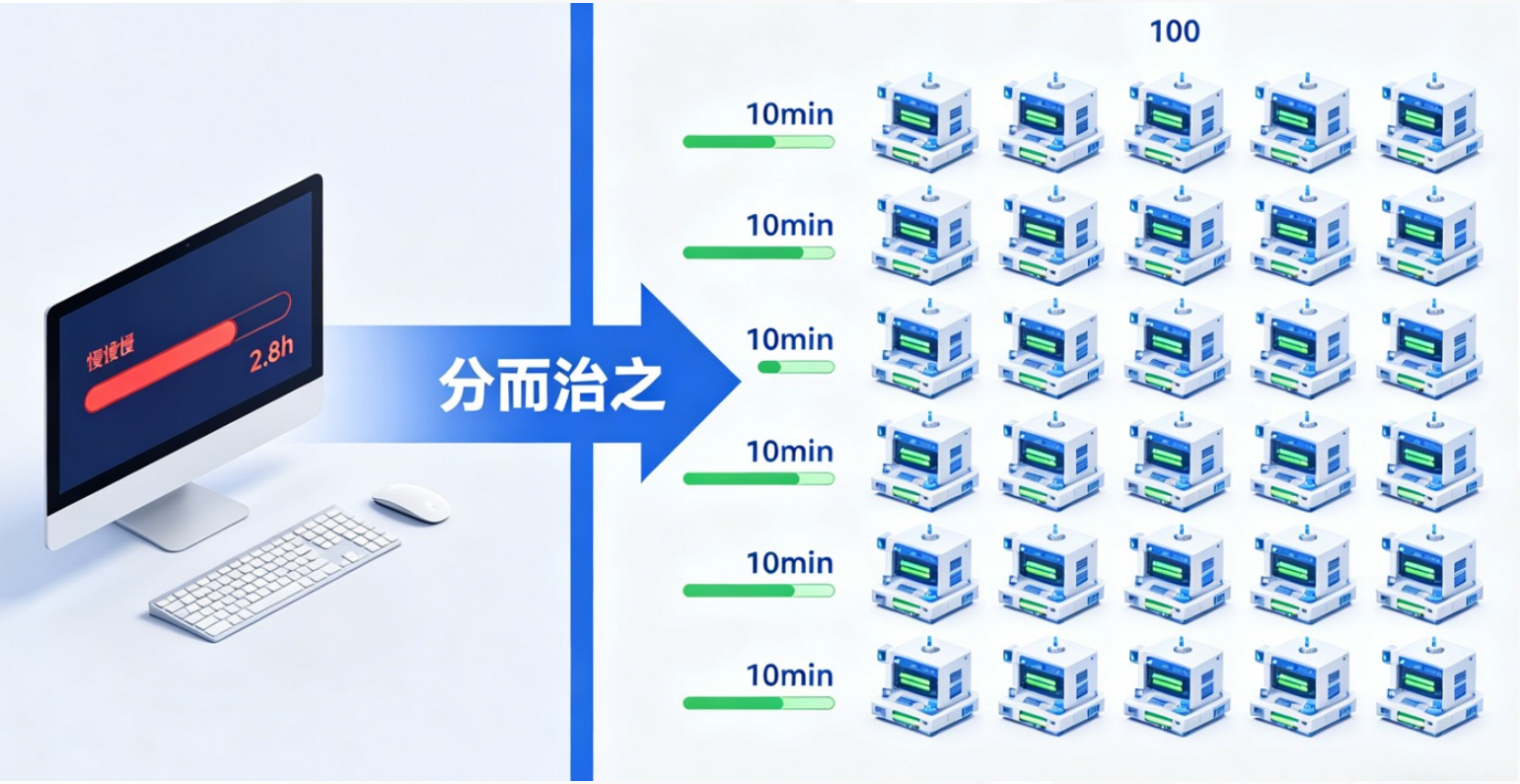
2000 → 2010 → 2025：6.2EB → 2ZB → 180ZB

单机硬盘 100MB/s × 1TB = 2.8 小时」

02 为什么需要并行计算?

西南交通大学 Southwest Jiaotong University

1TB ÷ 100 台 = 每台 10GB，数小时 → 10 分钟



03 并行计算的三大流派

西南交通大学 Southwest Jiaotong University

数据并行

征不同数据子集

征相同操作逻辑

代表技术





任务并行

数据特征相同/不同数据

操作特征不同操作逻辑

代表技术





流水线

数据特征源源不断的流式数据

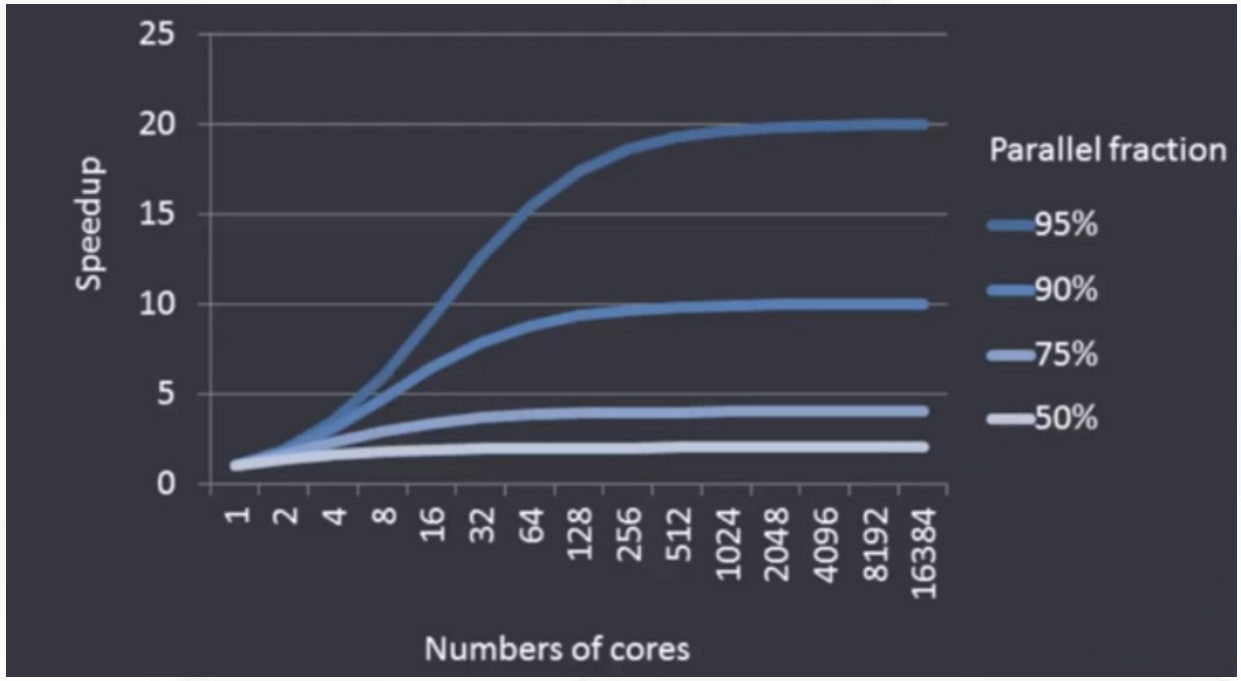
操作特征多阶段串联处理

代表技术



04 Amdahl定律

加速比 S(N) = 1 / (f + (1-f)/N)
*(*其中 f=串行部分比例，N=处理器数量*)*



即便 N→∞，加速比上限 = 1/f

05 Gustafson定律

西南交通大学 Southwest Jiaotong University

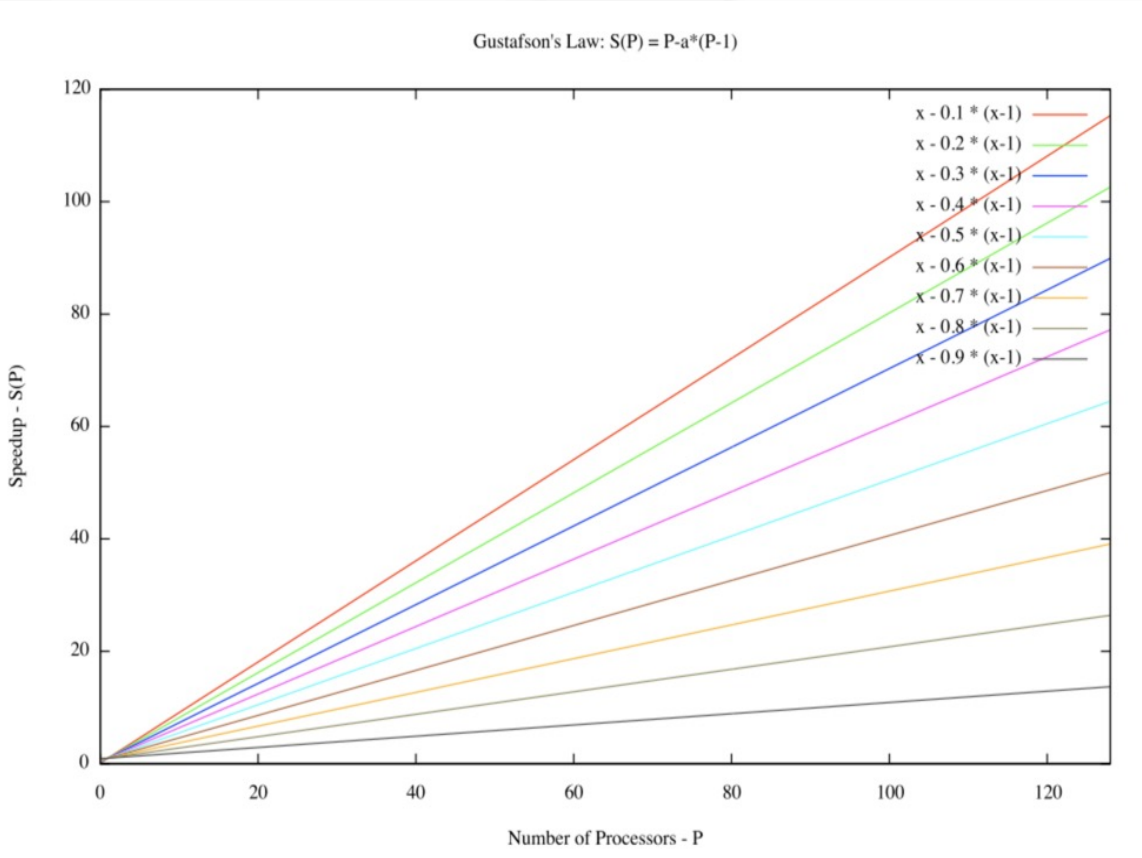
Amdahl定律的局限性

Amdahl假设：**问题规模是固定的。**
现实大数据场景：机器多了，我们要处理**更多的数据**（如100TB）！

Gustafson定律的核心主张

假设**执行时间固定**，随着处理器增多，系统可以处理更大规模的问题，加速比呈现近线性增长。

加速比 $S(N) = N - f \times (N - 1)$



06 Gustafson定律

西南交通大学 Southwest Jiaotong University

场景对比



Amdahl适用：实时/低延迟。



Gustafson 适用：大数据批处理

2 MapReduce编程模型

01 MapReduce的诞生与设计哲学

西南交通大学 Southwest Jiaotong University

历史坐标

2004年，Google在OSDI大会发表《MapReduce: Simplified Data Processing on Large Clusters》。它将复杂的分布式痛点（网络通信、容错、调度）全部封装到底层。



MapReduce: Simplified Data Processing on Large Clusters

Jeffrey Dean and Sanjay Ghemawat

jeff@google.com, sanjay@google.com

Google, Inc.

Abstract

MapReduce is a programming model and an associated implementation for processing and generating large data sets. Users specify a *map* function that processes a key/value pair to generate a set of intermediate key/value pairs, and a *reduce* function that merges all intermediate values associated with the same intermediate key. Many real world tasks are expressible in this model, as shown in the paper.

Programs written in this functional style are automatically parallelized and executed on a large cluster of commodity machines. The run-time system takes care of the details of partitioning the input data, scheduling the program's execution across a set of machines, handling machine failures, and managing the required inter-machine communication. This allows programmers without any experience with parallel and distributed systems to easily utilize the resources of a large distributed system.

Our implementation of MapReduce runs on a large cluster of commodity machines and is highly scalable: a typical MapReduce computation processes many terabytes of data on thousands of machines. Programmers find the system easy to use: hundreds of MapReduce programs have been implemented and upwards of one thousand MapReduce jobs are executed on Google's clusters every day.

1 Introduction

Over the past five years, the authors and many others at Google have implemented hundreds of special-purpose computations that process large amounts of raw data, such as crawled documents, web request logs, etc., to compute various kinds of derived data, such as inverted indices, various representations of the graph structure of web documents, summaries of the number of pages crawled per host, the set of most frequent queries in a given day, etc. Most such computations are conceptually straightforward. However, the input data is usually large and the computations have to be distributed across hundreds or thousands of machines in order to finish in a reasonable amount of time. The issues of how to parallelize the computation, distribute the data, and handle failures conspire to obscure the original simple computation with large amounts of complex code to deal with these issues.

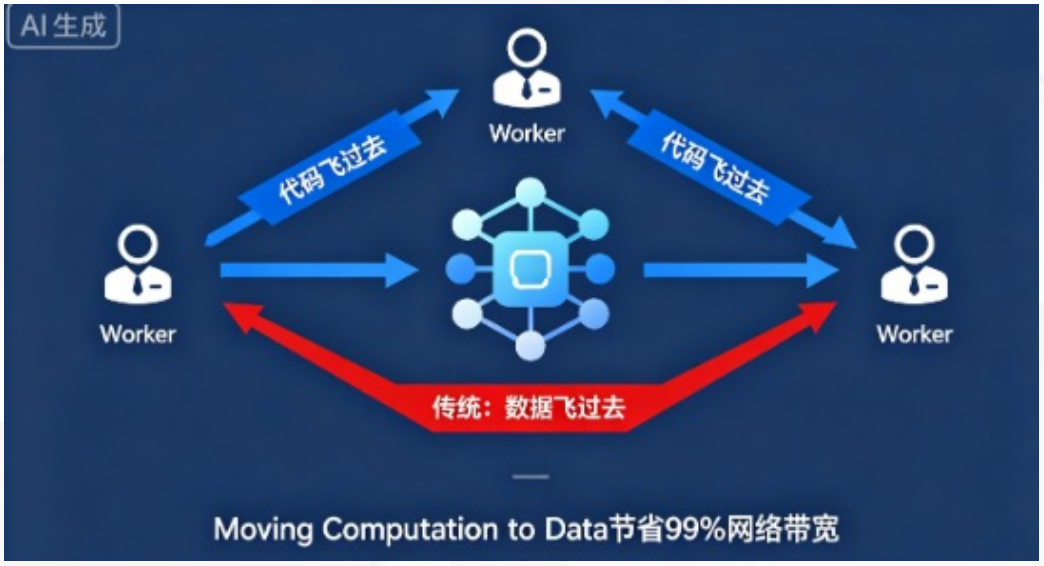
As a reaction to this complexity, we designed a new abstraction that allows us to express the simple computations we were trying to perform but hides the messy details of parallelization, fault-tolerance, data distribution and load balancing in a library. Our abstraction is inspired by the *map* and *reduce* primitives present in Lisp and many other functional languages. We realized that most of our computations involved applying a *map* operation to each logical "record" in our input in order to compute a set of intermediate key/value pairs, and then applying a *reduce* operation to all the values that shared the same key, in order to combine the derived data appropriately. Our use of a functional model with user-specified map and reduce operations allows us to parallelize large computations easily and to use re-execution as the primary mechanism for fault tolerance.

The major contributions of this work are a simple and powerful interface that enables automatic parallelization and distribution of large-scale computations, combined with an implementation of this interface that achieves high performance on large clusters of commodity PCs.

Section 2 describes the basic programming model and gives several examples. Section 3 describes an implementation of the MapReduce interface tailored towards our cluster-based computing environment. Section 4 describes several refinements of the programming model that we have found useful. Section 5 has performance measurements of our implementation for a variety of tasks. Section 6 explores the use of MapReduce within Google including our experiences in using it as the basis

To appear in OSDI 2004

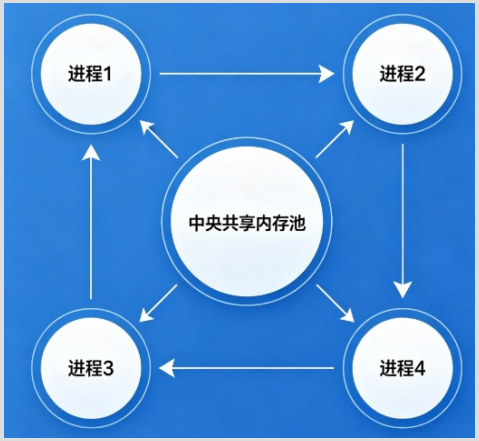
计算向数据移动（Moving Computation to Data）：不移动庞大的数据，而是把轻量的程序代码发送到数据所在的节点执行。省下巨大的网络带宽。



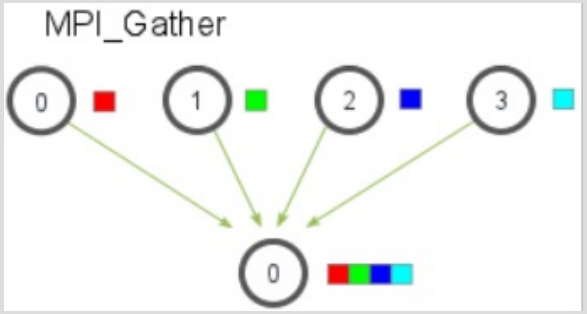
02 并行编程模型分类

西南交通大学 Southwest Jiaotong University

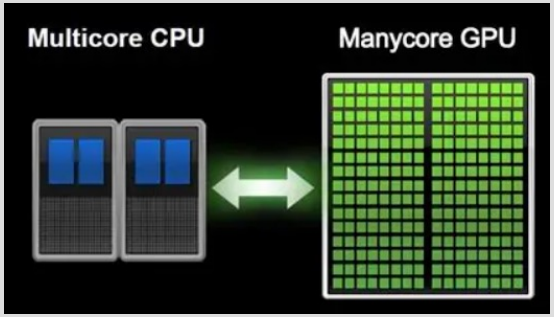
共享内存
（multiprocessing / OpenMP）



消息传递（MPI / mpi4py）



GPU 并行（CUDA）



03 Map与Reduce：数据变形

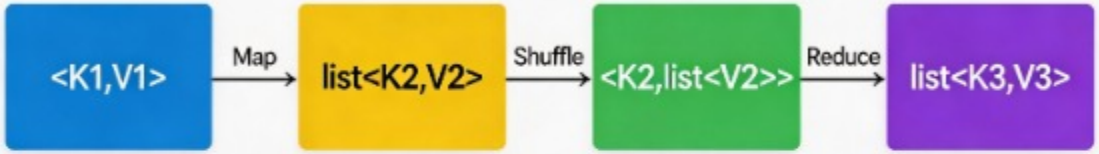
西南交通大学 Southwest Jiaotong University

Map函数：数据打散与重塑

输入：键值对 <K1, V1>
输出：零个或多个中间键值对 list(<K2, V2>)
动作：过滤、提取、转换。

Reduce函数：数据聚合

输入：中间键和它的值集合 <K2, list(V2)>
输出：最终结果 list(<K3, V3>)
动作：求和、求最值、拼接。



Map 过滤/提取/转换 |
Reduce 求和/求最值/拼接

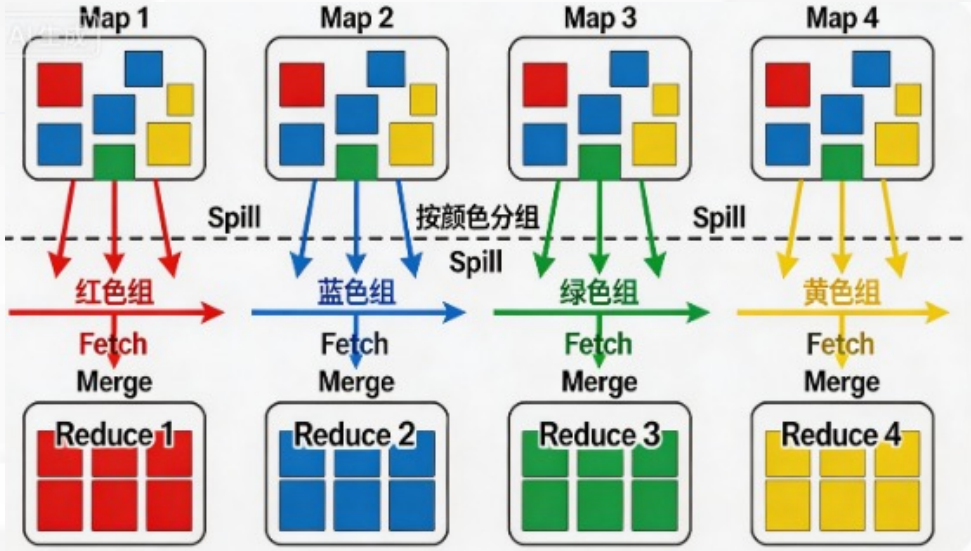
键值对（Key-Value）是实现**分组（Group By）**的通用语言。同一种Key必然被送到同一个Reduce节点。

04 Shuffle

西南交通大学 Southwest Jiaotong University

什么是Shuffle?

介于Map和Reduce之间的过程。它的唯一目的是：**保证所有Key相同的中间数据，都能被送到同一个Reduce任务中去。**



Shuple = MR 性能顿顿：
磁盘 I/O + 网络传输

Map端溢写（Spill）→ 网络拉取（Fetch）→ 归并排序（Merge）

05 Combiner

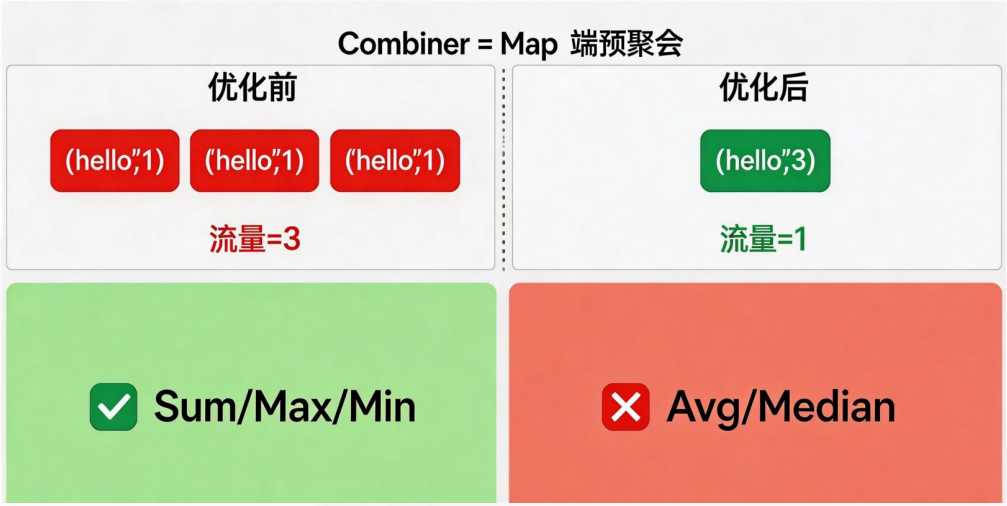
西南交通大学 Southwest Jiaotong University

为什么要引入Combiner?

既然网络传输是Shuffle的瓶颈，我们能否在数据离开Map节点前，**“就地”先合并一次**?

Combiner的作用

运行在Map端的“迷你Reduce”。
优化前Map输出：("hello",1), ("hello",1), ("hello",1) -> 传3条记录
Combiner优化后：("hello",3) -> 只传1条记录!



致命陷阱：滥用的代价

Combiner**绝不能改变最终结果**。它只适用于满足**交换律**和**结合律**的操作。
适用：Sum（求和）、Max（最大值）
不适用：Average（平均值），因为 Avg(Avg(1,3), 5) ≠ Avg(1,3,5)

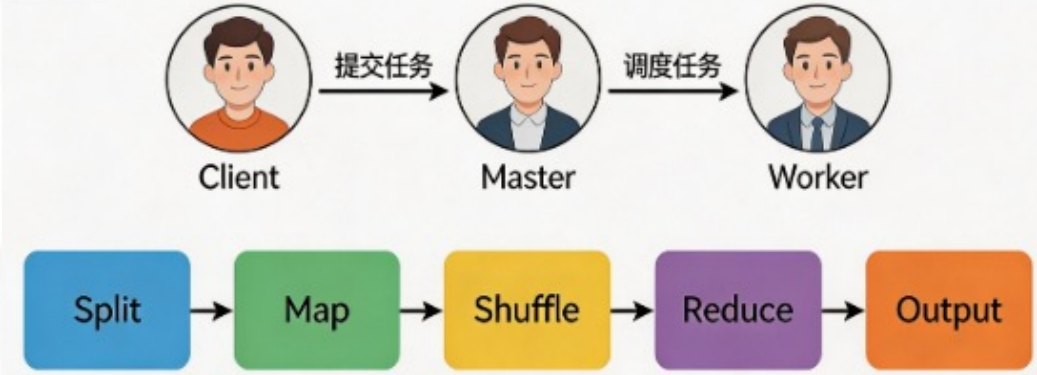
3 MapReduce执行流程

01 MapReduce整体执行生命周期

西南交通大学 Southwest Jiaotong University

核心角色与分工

1. **Client**：用户提交作业的起点。
2. **Master（JobTracker/AppMaster）**：大管家，负责拆分任务、调度和监控。
3. **Worker（TaskTracker/NodeManager）**：打工人，真正执行Map和Reduce代码的物理机。



数据的五步走

InputSplit：1TB文件按128MB切块（分片）。
Map阶段：每个分片启动一个Map Task执行。
Shuffle阶段：数据按规则在集群中大迁徙。
Reduce阶段：接收合并后的数据执行聚合。
Output阶段：最终结果写入HDFS文件系统。

02 容错机制（Fault Tolerance）

大规模集群中，**硬件故障不是意外，而是常态。**

为什么MR容错做得这么好?

无状态计算（Shared-Nothing）：Map和Reduce互不干扰，失败了重跑就行。

中间结果落盘：Map的结果写入本地磁盘，而不是直接流向Reduce。



03 推测执行（Speculative Execution）

西南交通大学 Southwest Jiaotong University

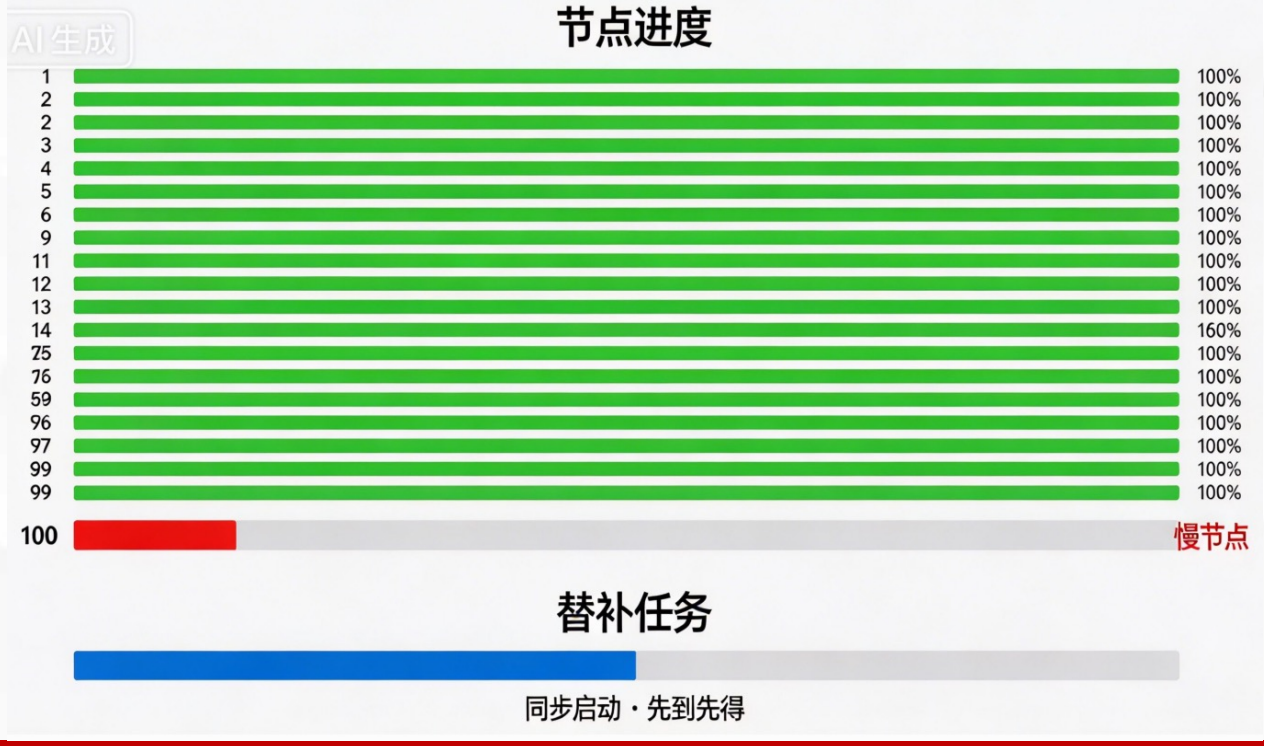
拖后腿的“长尾效应”

100个任务中，99个都只花了1分钟，偏偏最后1个任务因为所在机器硬盘老化，跑了10分钟。整个作业的时间就被这1个人拖累成了10分钟！

应对方案：找个替补

当Master发现某个Task进度**远低于**平均水平时：

1. 不去停掉这个慢任务。
2. 在其他空闲机器上，**同时启动一个一模一样的“替补任务”**。
3. 哪个先跑完，就用哪个的结果，并立刻杀掉另一个。



04 推测执行（Speculative Execution）

西南交通大学 Southwest Jiaotong University

思考：任何任务都可以开启推测执行吗？（提示：如果任务涉及到写数据库呢？）



A、可以

B、不能

4 MapReduce编程模式

01 计数与汇总（以Word Count为例）

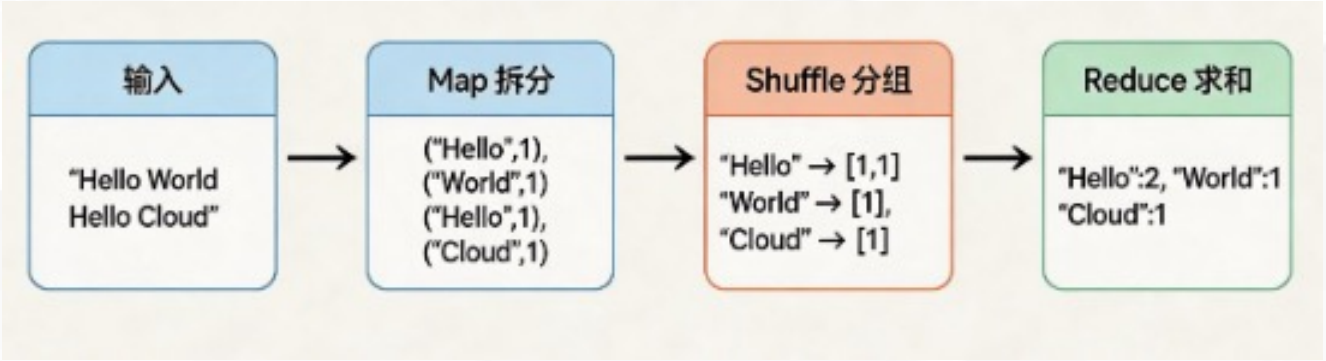
西南交通大学 Southwest Jiaotong University

业务场景

网页PV统计、日志错误类型统计、词频分析。

核心逻辑解析

Map端：拆分并打标签。将一条包含多个单词的句子，拆解为多个独立的单词，并为每个单词附上数字`1`。
Reduce端：累加求和。接收同一个单词的所有`1`，进行加法运算。



02 计数与汇总（以Word Count为例）

西南交通大学 Southwest Jiaotong University

核心代码片段展示

```
# 核心 Map 逻辑
def map_function(document):
    for word in document.split():
        emit(word, 1)  # 输出格式：(Key, Value)

# 核心 Reduce 逻辑
def reduce_function(word, counts_list):
    total = sum(counts_list)  # [1, 1, 1...] 求和
    emit(word, total)
```

(注：框架会自动将Map输出的`1`聚合为`counts_list`)

03 倒排索引（Inverted Index）

西南交通大学 Southwest Jiaotong University

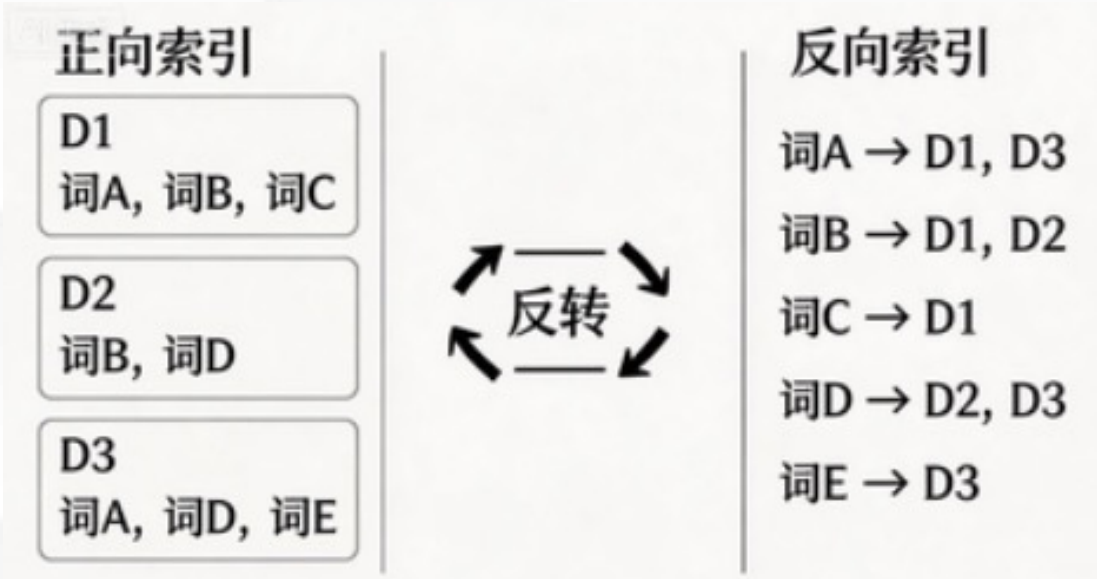
业务场景

搜索引擎的底层基石。将“**文档包含哪些词**”的正向结构，反转为“**这个词出现在哪些文档**”。

核心逻辑解析

Map端：读取文档内容，以“**单词**”为Key，以“**文档ID**”为Value进行输出。

Reduce端：将属于同一个单词的所有文档ID收集起来，去重并排序，形成“拉链”（Posting List）。



04 倒排索引（Inverted Index）

西南交通大学 Southwest Jiaotong University

核心代码片段展示

```
# 核心 Map 逻辑：正向切分
def map_function(doc_id, text_content):
    words = unique(text_content.split()) # 去重避免单文档多次记录
    for word in words:
        emit(word, doc_id)  # 反转：Word变成Key

# 核心 Reduce 逻辑：拼接拉链
def reduce_function(word, doc_ids_list):
    sorted_docs = sort(unique(doc_ids_list))
    emit(word, sorted_docs)
```

05 排序与Top N

西南交通大学 Southwest Jiaotong University

业务场景

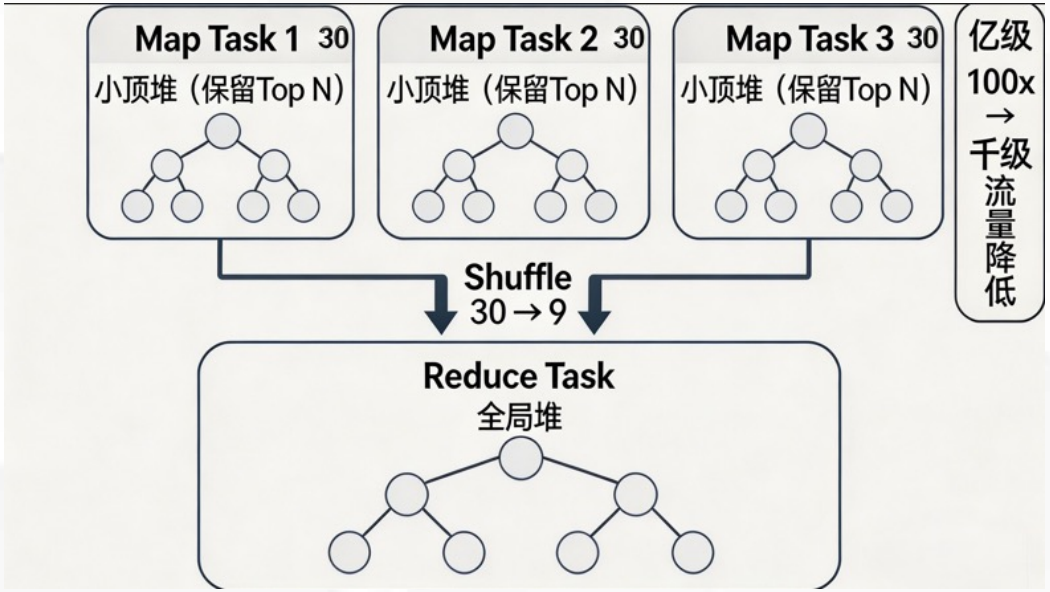
热门商品排行榜、高频搜索词Top 10。在海量数据中找出最大/最小的N条。

核心思想：局部最优 -> 全局最优

如果所有数据全丢给一个Reduce去排序，会撑爆内存！

解决之道：

1. **Map端发力**
2. **数据骤降**
3. **Reduce收尾**



核心逻辑说明

不拼全量数据，只拼局部精英。大幅度削减了网络洗牌（Shuffle）的压力。

06 排序与Top N

西南交通大学 Southwest Jiaotong University

核心代码片段展示

```
# ── Map：每个节点只保留局部 TopN，大幅减少 Shuffle 传输量 ──
def topn_map(shard, N):
    heap = []
    for item, score in shard:
        heapq.heappush(heap, (score, item))
        if len(heap) > N:
            heapq.heappop(heap)  # 弹出最小值，堆中始终保留最大的 N 个
    return heap
# ── Reduce：汇总所有局部冠军，选出全局 TopN ──
def topn_reduce(candidates, N):
    return heapq.nlargest(N, candidates)  # 从所有局部 TopN 中选全局 TopN
```

07 排序与Top N

西南交通大学 Southwest Jiaotong University

核心代码片段展示

```
# ── 执行 ──
N = 3
shards = [
    [("耳机", 1200), ("数据线", 300), ("手机壳", 500)],   # Map 分片 0
    [("手表", 2500), ("充电宝", 800), ("支架",  150)],   # Map 分片 1
    [("TWS", 4000),  ("键盘",  950), ("鼠标垫", 600)],   # Map 分片 2
]
local_tops = [topn_map(s, N) for s in shards]          # 各分片并行取局部 TopN
all_candidates = [item for top in local_tops for item in top]  # Shuffle 汇聚
result = topn_reduce(all_candidates, N)

for rank, (score, name) in enumerate(result, 1):
    print(f"第{rank}名: {name}  浏览量={score}")
```

```
# 预期输出
第1名: TWS  浏览量=4000
第2名: 手表  浏览量=2500
第3名: 耳机  浏览量=1200
```

08 表关联（Join操作）

大数据中最头疼的问题：如何将分散在两个大文件（如用户表、订单表）的数据对应起来？

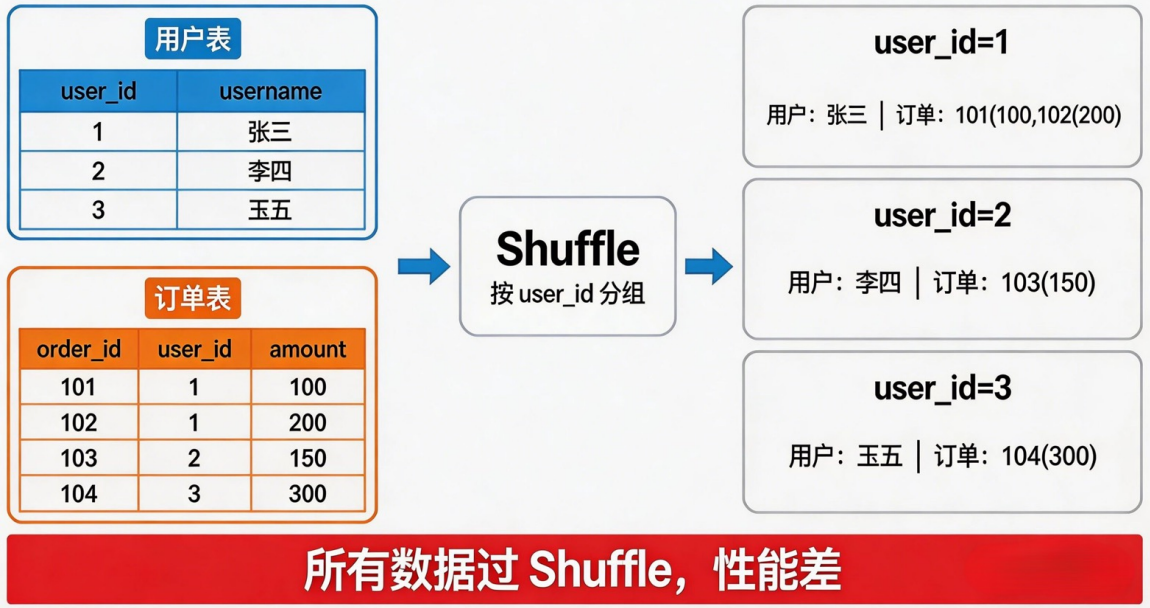
Reduce-Side Join（通用但笨重）

思路：利用Shuffle的按Key分组机制。

Map端：分别为两个表的数据打上标签（如Tag="User"和Tag="Order"），统一以关联键（User_ID）作为Key输出。

Reduce端：相同User_ID的数据相聚，再根据标签拆分拼接。

缺点：所有数据都要经过Shuffle，极其消耗性能。



09 表关联（Join操作）

西南交通大学 Southwest Jiaotong University

```
users  = [(1,"Alice"), (2,"Bob"), (3,"Charlie")]
orders = [(1,"手机"),  (1,"图书"), (2,"运动鞋"), (3,"笔记本")]
# == Reduce-Side Join：两表都以 user_id 为 Key，Shuffle 自动聚合 ==
# Map：打 tag 标明来源，以 user_id 为 Key
def join_map(tag, uid, info):
    return (uid, (tag, info))  # Key=user_id，Value=(表名, 内容)
# Shuffle：按 user_id 分组（框架自动完成）
grouped = defaultdict(list)
for uid, name    in users:  grouped[uid].append(join_map("user",  uid, name))
for uid, product in orders: grouped[uid].append(join_map("order", uid, product))
# Reduce：按 tag 分拣，拼接用户与订单
for uid, records in grouped.items():
    user = next(info for _, (tag, info) in records if tag == "user")
    items = [info for _, (tag, info) in records if tag == "order"]
    print(f"用户{uid} {user} 购买了 {items}")
```

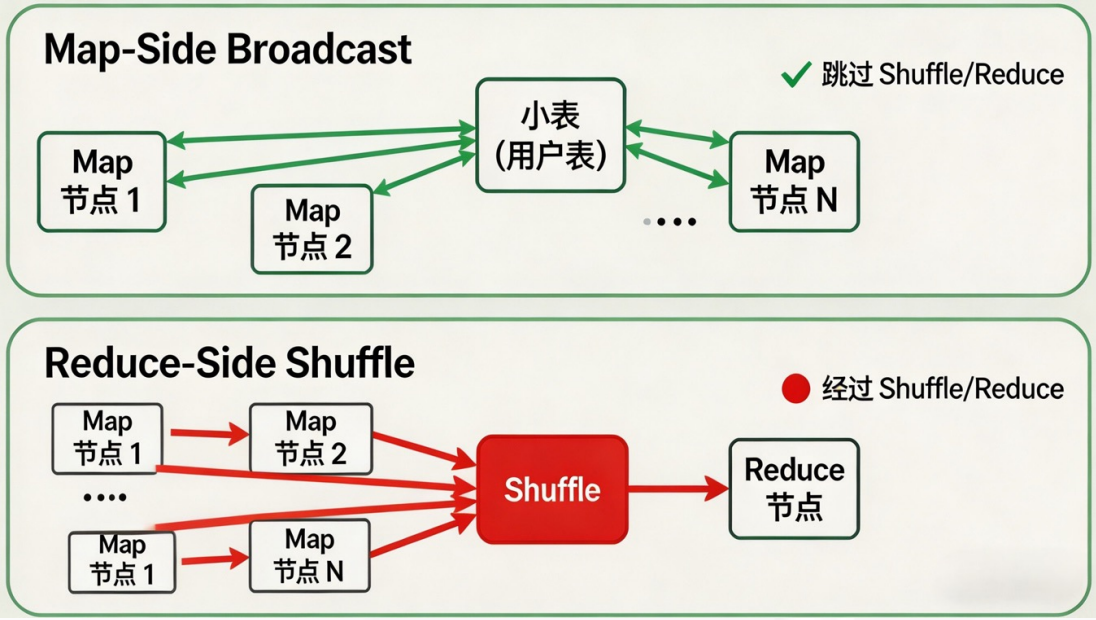
核心代码片段展示

10 表关联（Join操作）

西南交通大学 Southwest Jiaotong University

Map-Side Join（轻巧但挑剔）

思路：当存在“一大一小”两张表时使用。
机制：将“小表”通过分布式缓存（Distributed Cache）广播到所有Map节点的内存中。
执行：Map在处理“大表”的每一行时，直接查本地内存中的“小表”进行合并。**完全跳过Shuffle和Reduce阶段！**



11 表关联（Join操作）

西南交通大学 Southwest Jiaotong University

核心代码片段展示

```
# == Map-Side Join：小表广播到内存，完全跳过 Shuffle ==
user_dict = dict(users)  # 用户表（小表）加载到每个 Map 节点内存
for uid, product in orders:
    print(f"{user_dict[uid]} 购买了 {product}")
```

```
# 预期输出
用户1 Alice 购买了 ['手机', '图书']
用户2 Bob 购买了 ['运动鞋']
用户3 Charlie 购买了 ['笔记本']

Alice 购买了 手机
Alice 购买了 图书
Bob 购买了 运动鞋
Charlie 购买了 笔记本
```

5 Hadoop MapReduce实践

01 Hadoop生态

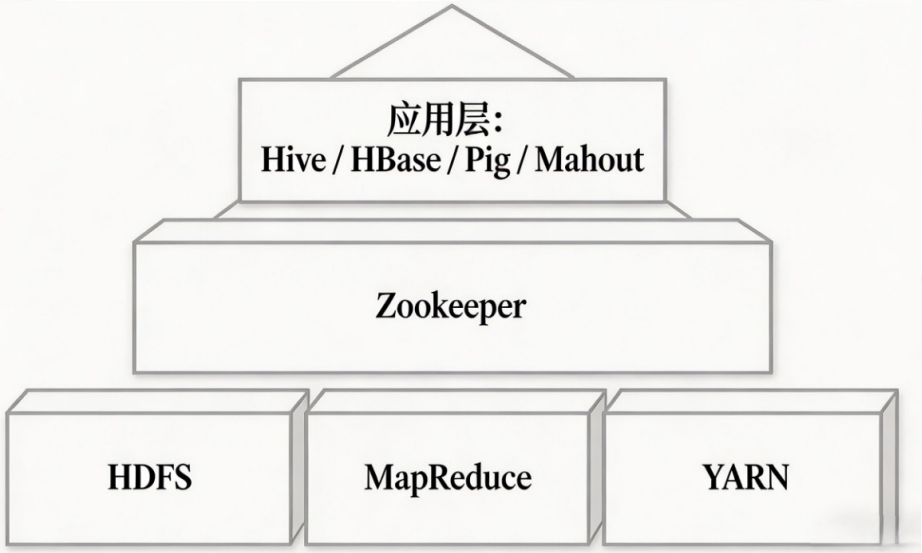
Hadoop不仅是一个软件，更是一个包含存储、计算、调度的庞大生态圈。

三驾马车

1. **HDFS（分布式文件系统）**2. **MapReduce（分布式计算）**3. **YARN（分布式资源调度）**

繁荣的上层建筑

Hive，HBase，Zookeeper



02 资源大管家：YARN

西南交通大学 Southwest Jiaotong University

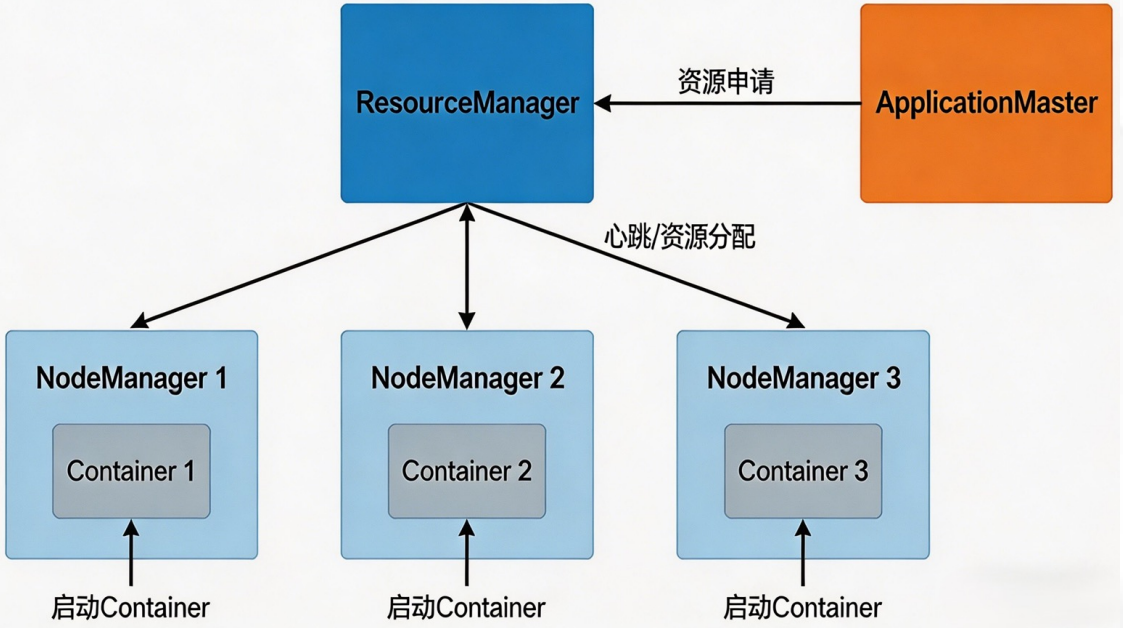
YARN核心架构

ResourceManager (RM)：集群的全局资源CEO，掌握所有节点的CPU和内存情况。

NodeManager (NM)：每台机器上的工头，负责本机的资源汇报与容器管理。

ApplicationMaster (AM)：每个作业专属的项目经理，负责向RM申请资源，并督促NM执行任务。

Container（容器）：资源的最小抽象单位（例如：2核CPU+4GB内存）。



03 YARN的资源调度策略

西南交通大学 Southwest Jiaotong University

集群资源有限，多个部门抢资源怎么办？

1. FIFO Scheduler（先进先出）

机制：谁先提交，谁占用全部资源，排队执行。
缺点：大作业会“饿死”排在后面的紧急小作业。

2. Capacity Scheduler（容量调度器）

机制：多租户共享。预先划分队列（如：生产组60%，测试组40%）
亮点：资源借用机制。如果测试组暂时没任务，生产组可以借用这40%的资源；一旦测试组来任务了，生产组必须归还。

3. Fair Scheduler（公平调度器）

机制：动态平分资源。1个人跑占100%，2个人跑各占50%。保障每个人都能得到一定资源。

6 MapReduce的局限与演进

01 为什么我们觉得MapReduce变慢了？

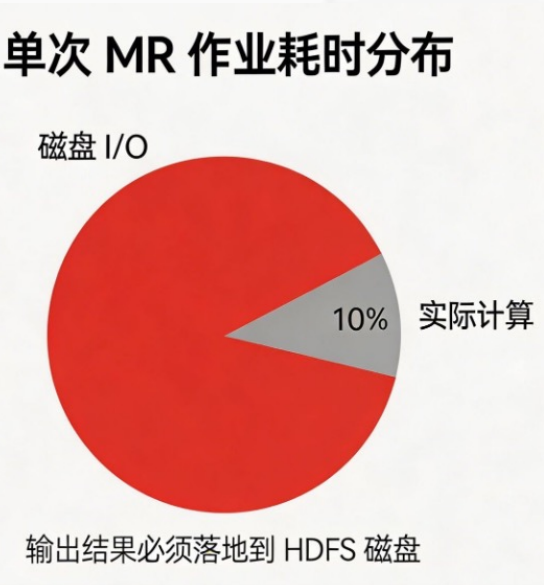
西南交通大学 Southwest Jiaotong University

致命伤：对磁盘的高度依赖

MapReduce的每一次作业，输出结果**必须落地到HDFS磁盘。**

痛点场景：机器学习迭代计算

假设训练一个K-Means聚类模型，需要迭代100次。





1. 表达能力有限：只有 Map 和 Reduce，复杂逻辑需串联几十个 MR 作业。
2. 高延迟：JVM 启动开销大，无法用于交互式秒级查询。
3. 只能做批处理：对流式实时数据束手无策。

02 Apache Spark的降维打击

西南交通大学 Southwest Jiaotong University

MR 磁盘 VS Spark 内存 数据流对比



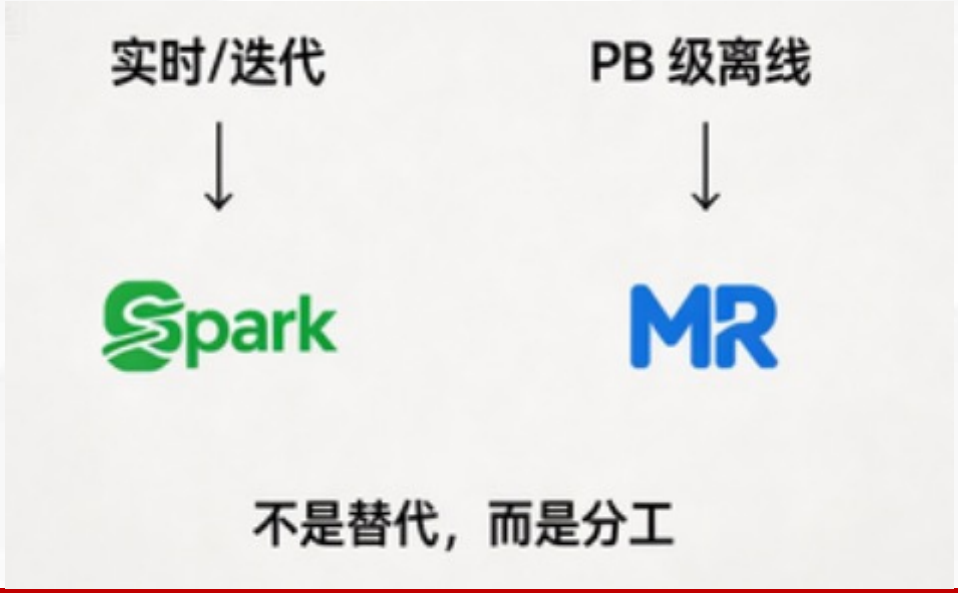
维度	MapReduce	Apache Spark
中间数据	写入磁盘 (慢)	保留内存 (极快)
编程范式	仅Map/Reduce	map, filter, groupBy, join...
执行速度	基础基准	逻辑回归等迭代场景快 100倍+
应用场景	离线批处理	批处理、SQL、流计算、图计算

03 Apache Spark的降维打击

西南交通大学 Southwest Jiaotong University

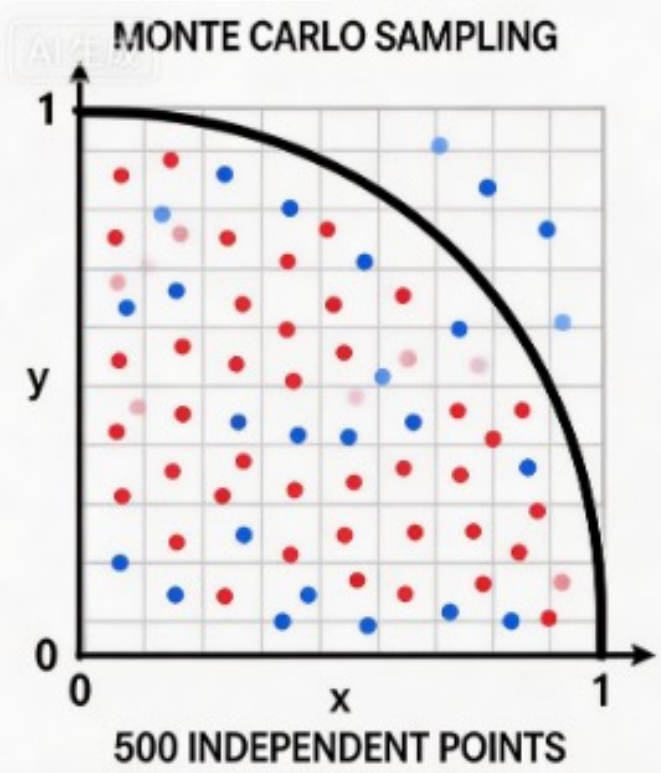
MapReduce真的淘汰了吗?

并没有。对于**超大规模（PB级别）且对时间不敏感**的离线清洗任务，MapReduce凭借无敌的稳定性和容错性，依然在众多大厂底层默默服役。



7 应用案例与演示代码

01 Demo ① 蒙特卡洛求 π 原理



ESTIMATION FORMULA

$$P(in) = \frac{\pi}{4}$$

$$\pi \approx 4 \times \frac{N_{in}}{N_{total}}$$

INDEPENDENT PARALLEL SAMPLING

Demo ① MPI 通信原语

---- MPI HALF CHEAT SHEET ---

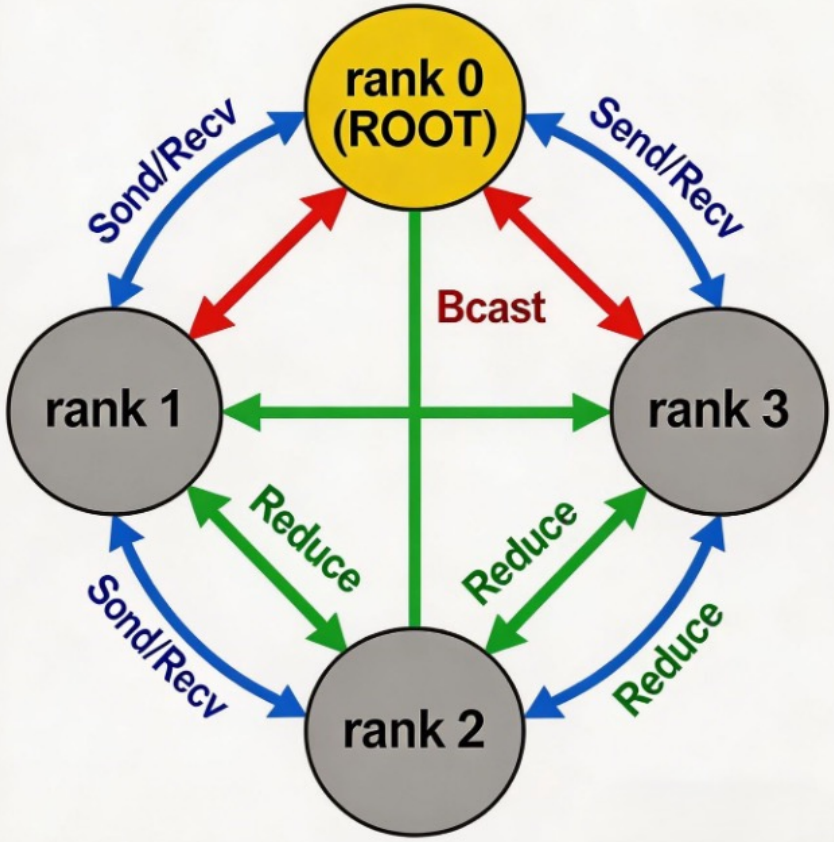
1. MPI_Init/Finalize:
Initialize MPI environment / Clean up MPI resources

2. MPI_Comm_rank/size:
Get own process ID (rank) / Total number of processes

3. MPI_Bcast:
Send data from root to ALL processes

4. MPI_Reduce:
Aggregate data from ALL processes to root

RING TOROLOGY



03 Demo ① 完整代码

```
# pi_mpi.py —— 运行：mpiexec -n 4 python pi_mpi.py
from mpi4py import MPI
import random

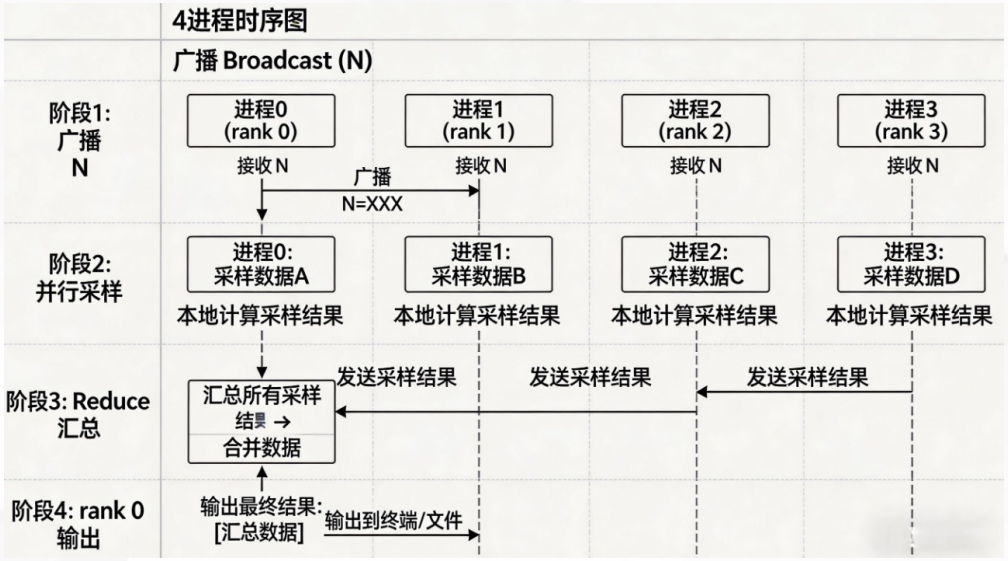
comm = MPI.COMM_WORLD
rank, size = comm.Get_rank(), comm.Get_size()

N_TOTAL = 10_000_000
N_LOCAL = N_TOTAL // size

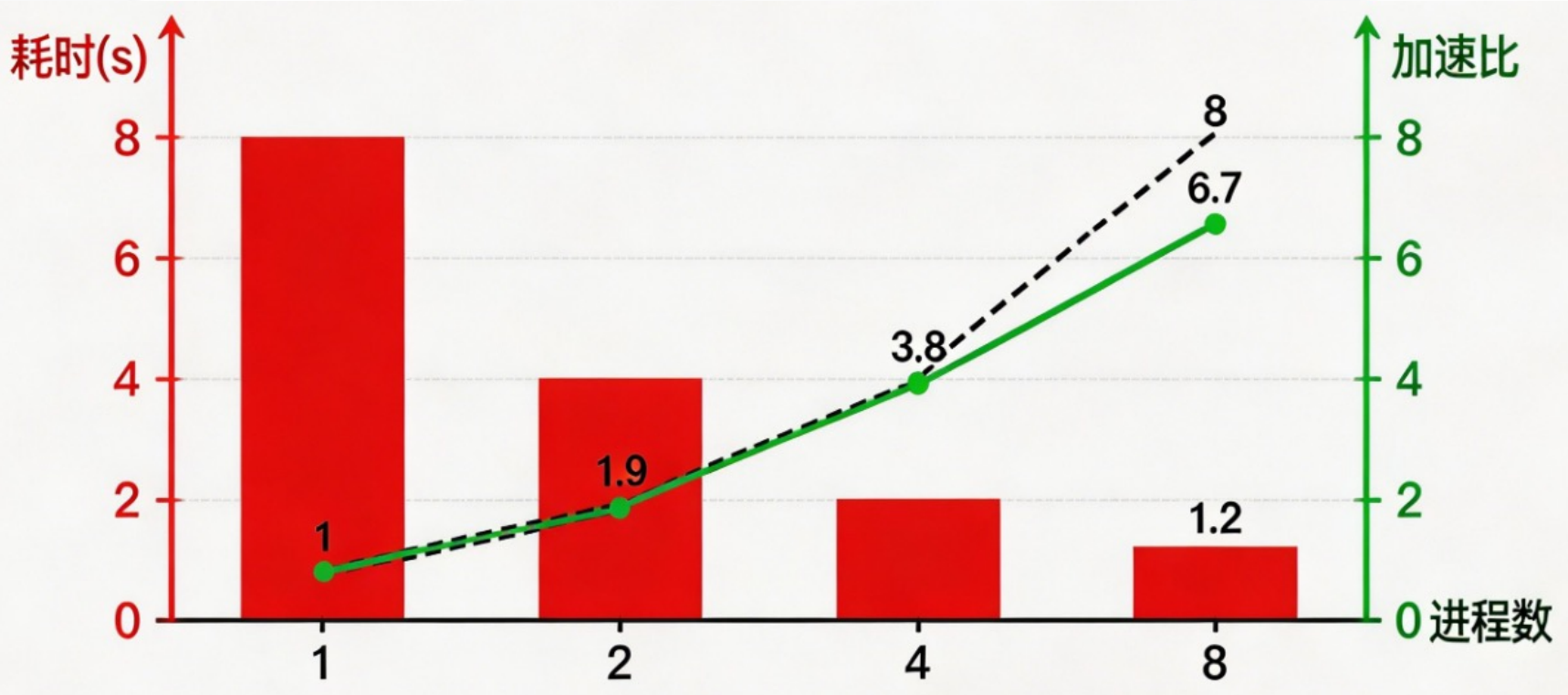
random.seed(rank * 42)
hits = sum(1 for _ in range(N_LOCAL)
           if random.random()**2 + random.random()**2 <= 1.0)

total_hits = comm.reduce(hits, op=MPI.SUM, root=0)

if rank == 0:
    pi_est = 4.0 * total_hits / N_TOTAL
    print(f"π ≈ {pi_est:.6f}")
```



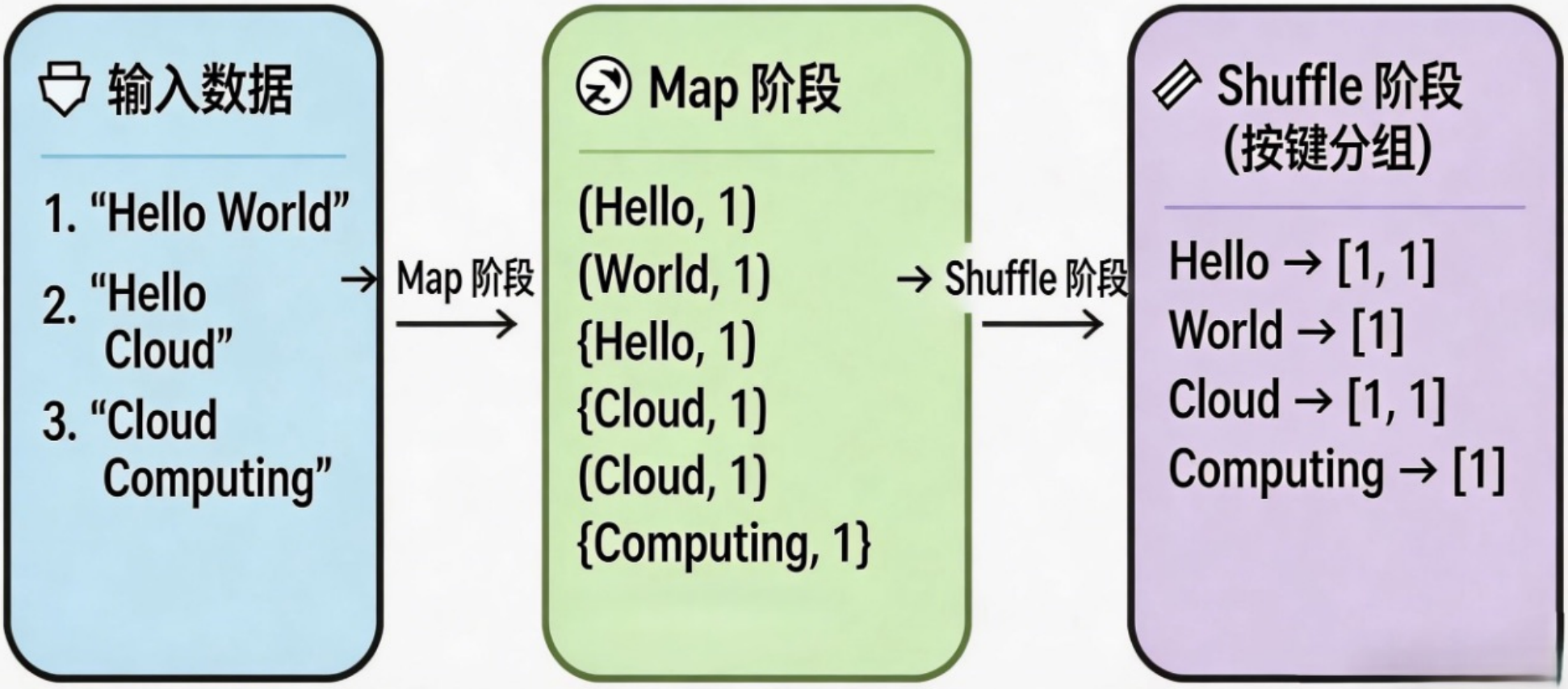
04 Demo ① 实测加速比



实测 6.7x（理想 8x），效率 84%

为什么没到 8x？通信开销 + 主进程汇总开销

05 Demo ② WordCount 三阶段



06 Demo ② 完整代码

西南交通大学 Southwest Jiaotong University

```
# wordcount_mr.py —— 纯 Python 模拟 MapReduce
from collections import defaultdict

def split_input(documents, num_shards=3):
    return [documents[i::num_shards] for i in range(num_shards)]

def map_phase(shard):
    out = []
    for doc in shard:
        for word in doc.lower().split():
            out.append((word, 1))
    return out

def shuffle_phase(map_outputs):
    grouped = defaultdict(list)
    for partial in map_outputs:
        for k, v in partial:
            grouped[k].append(v)
    return dict(grouped)

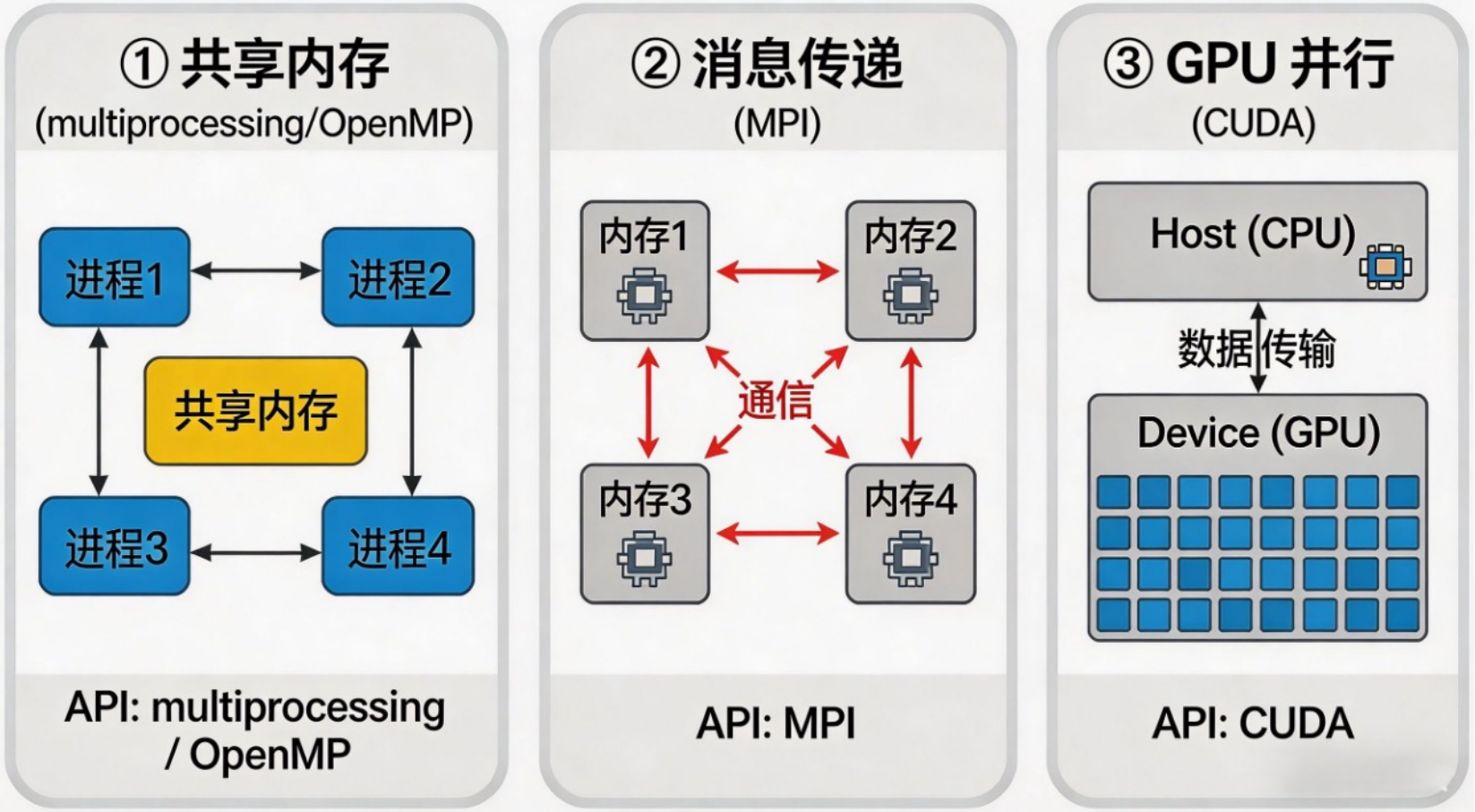
def reduce_phase(grouped):
    return {k: sum(vs) for k, vs in grouped.items()}

docs   = ["Hello World", "Hello Cloud", "Cloud Computing is great"]
shards  = split_input(docs)
map_out = [map_phase(s) for s in shards]
grouped = shuffle_phase(map_out)
final   = reduce_phase(grouped)
print(final)
# {'hello':2, 'world':1, 'cloud':2, 'computing':1, 'is':1, 'great':1}
```

07 Demo ② 数据倾斜挑战



Demo ③ 共享内存 vs 消息传递



09 Demo ③ 完整代码

西南交通大学 Southwest Jiaotong University

```
# speedup_mp.py —— multiprocessing 加速比对比
from multiprocessing import Pool
import time, math

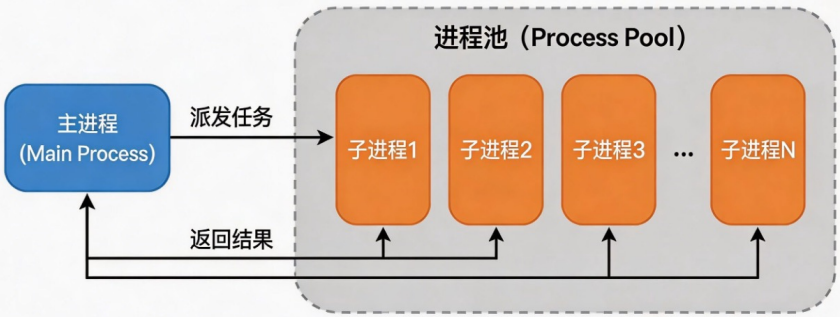
def heavy(chunk):
    return [math.sqrt(x*x + math.sin(x)) for x in chunk]

def benchmark(n, chunks):
    t0 = time.time()
    if n == 1:
        [heavy(c) for c in chunks]
    else:
        with Pool(n) as pool:
            pool.map(heavy, chunks)
    return time.time() - t0
```

```
N = 10_000_000
data = list(range(N))
chunks = [data[i:i+N//8] for i in range(0, N, N//8)]

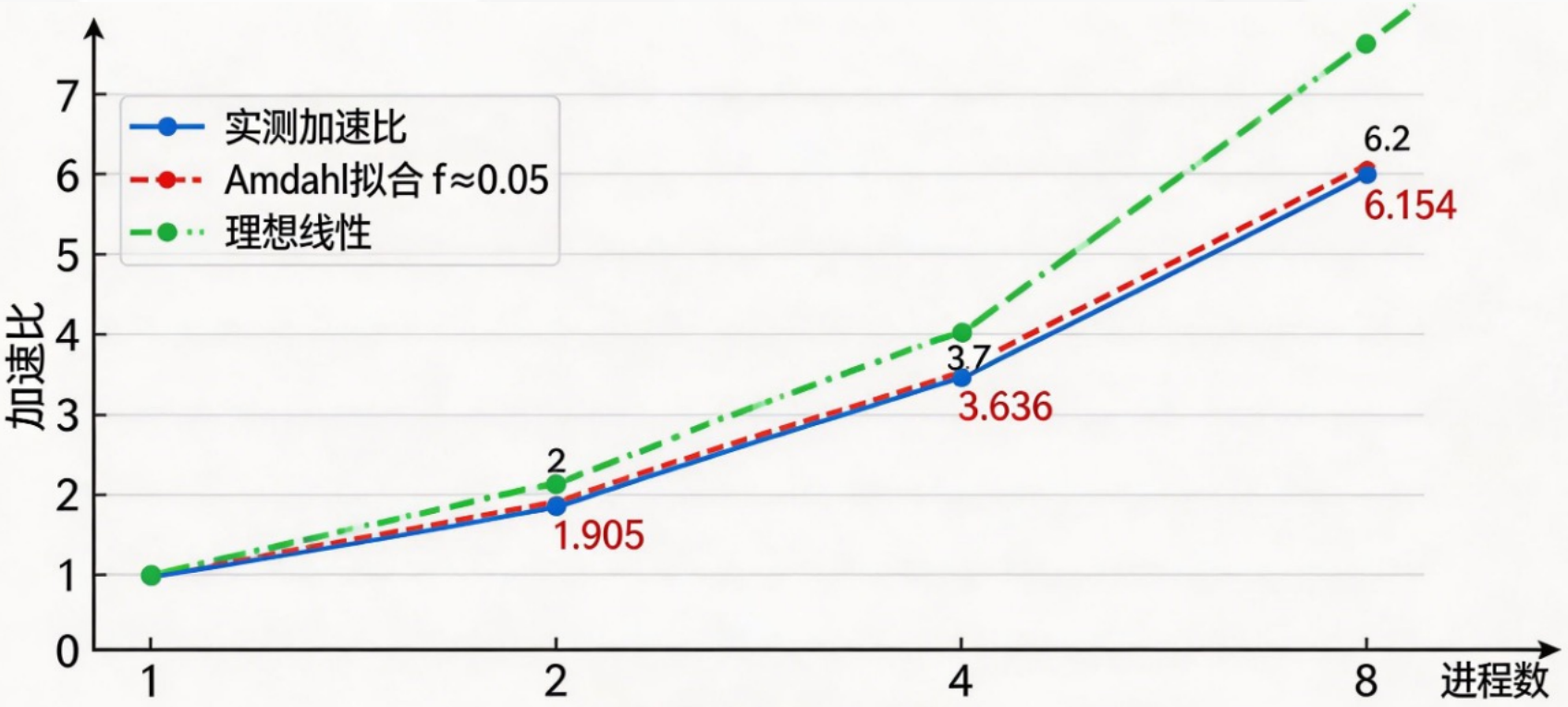
t1 = benchmark(1, chunks)
print(f"{'workers':<8}{'time(s)':<10}{'speedup':<10}")
for w in [1, 2, 4, 8]:
    t = benchmark(w, chunks)
    print(f"{w:<8}{t:<10.2f}{t1/t:<10.2f}")
```

进程池调度示意图



主进程管理N个子进程，统一派发任务、回收执行结果

Demo ③ 实测 + Amdahl 拟合



实测 8 进程 = 6.2x

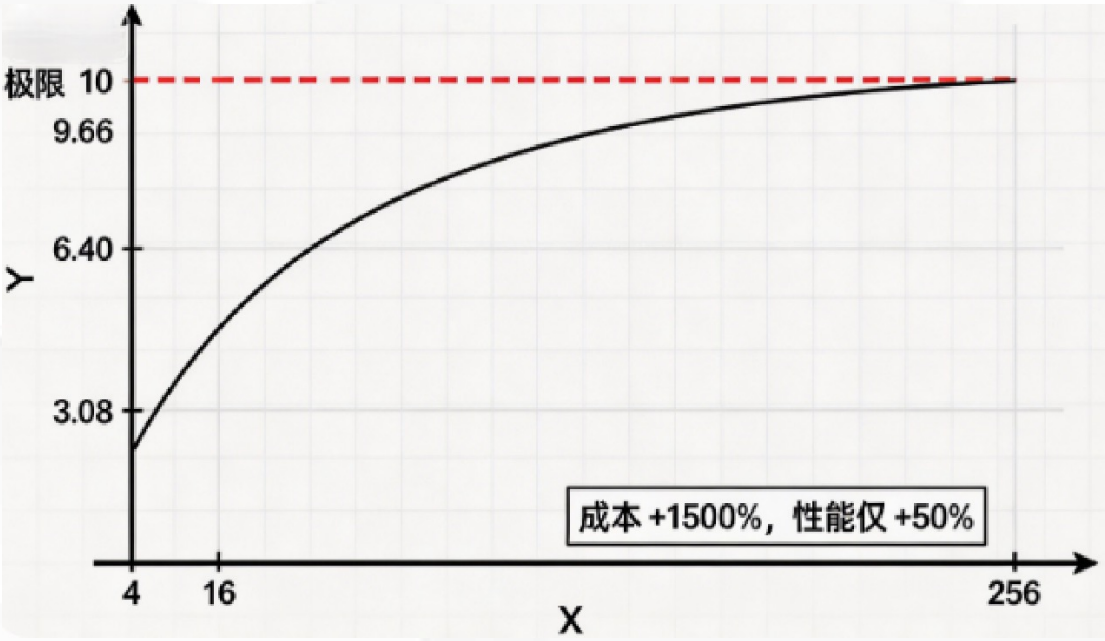
Amdahl 拟合 f≈5%

优化方向：找出那 5% 串行代码

11 Amdahl 代码推演

```
# 串行比例 f, 处理器数量 n
def amdahl_speedup(f, n):
    return 1.0 / (f + (1 - f) / n)

# 假设代码中存在10%的不可并行部分 (f=0.1)
print(amdahl_speedup(0.1, n=4))   # 4核加速比: 3.08倍
print(amdahl_speedup(0.1, n=16))  # 16核加速比: 6.40倍
print(amdahl_speedup(0.1, n=256)) # 256核加速比: 9.66倍
print(amdahl_speedup(0.1, n=无穷大)) # 理论极限: 10倍
```



12 Demo 三方案横向对比

西南交通大学 Southwest Jiaotong University

维度	单机串行	multiprocessing	mpi4py
通信	—	共享内存	显式 Send/Recv
部署成本	0	0（标准库）	需装 MPI
跨机器	❌	❌	✅
适合场景	调试	单机多核	集群

13 课堂互动与思考题

西南交通大学 Southwest Jiaotong University

随堂头脑风暴

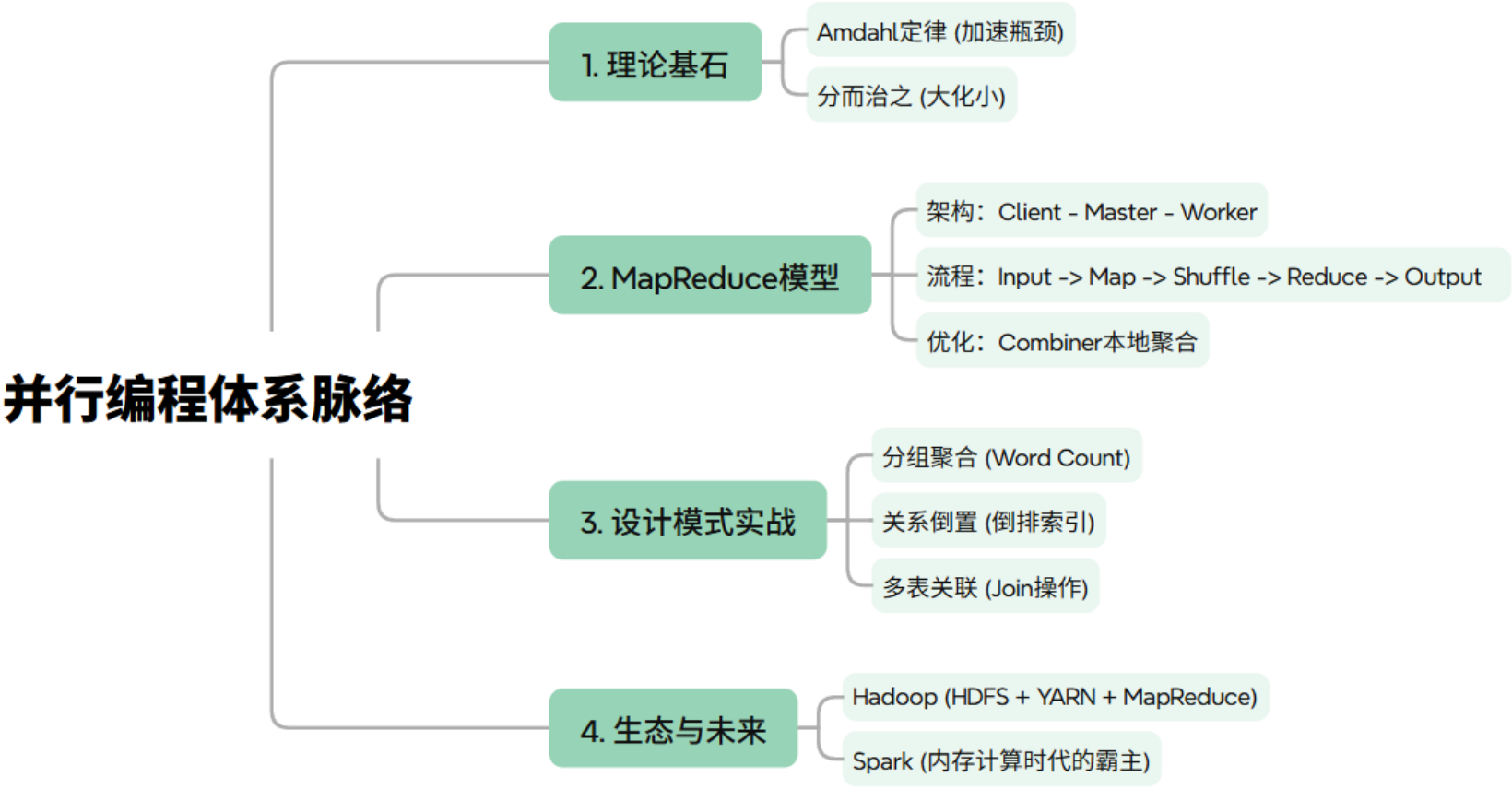
1. **Amdahl计算**：一个程序串行部分占20%，使用8个处理器时加速比是多少？如果要达到5倍加速，至少需要多少个处理器？串行部分最多能占多少？
2. **Combiner诊断**：为什么计算中位数（Median）不能直接使用Combiner？
3. **数据倾斜痛点**：在Word Count中，如果某个词（如英语中的"the"）出现频率远超其他词，会导致某个Reduce任务疯狂加班，而其他Reduce早早休息。这种情况该如何优化？
4. **框架抉择**：如果你是一家创业公司的CTO，要做实时的用户推荐系统，你会选择MapReduce还是Spark？为什么？

14 本讲知识要点总复习

西南交通大学 Southwest Jiaotong University

核心模块	必须掌握的关键点
并行计算理念	分而治之，移动计算而非移动数据
理论极限	Amdahl定律（限制上限），Gustafson定律（规模扩展）
MapReduce核心	Map(散列) -> Shuffle(归组/排序) -> Reduce(聚合)
网络优化	Combiner（前提：满足交换律与结合律）
容错调度	Task重试、推测执行解决长尾问题
生态演进	YARN分离调度，Spark以内存计算打破MR的磁盘I/O魔咒

15 并行编程模型全局知识图谱



16 下次课预告 & 课后作业

西南交通大学 Southwest Jiaotong University

知识点1：RDD
Spark弹性分布式数据集（RDD）

知识点2：算子
Transformation与Action算子的延迟计算机制

知识点3：DAG
内存计算背后的DAG原理

课后作业1：
假设某算法串行部分占比15%，请分别计算并绘制在8核、16核、32核下的Amdahl与Gustafson加速比折线图

课后作业2：
不写代码，只用自然语言描述——如何用MapReduce统计一份社交网络好友关系表（A是B的好友），找出所有用户的**共同好友**？（提示：思考Map的Key应该输出什么组合？）

17 Q & A

西南交通大学 Southwest Jiaotong University

谢谢！欢迎提问与讨论